

Implement a proper copy constructor

Absolutely. This is a **very common C++ OOP interview question**, especially when pointers/dynamic memory are involved.

Deep Copy vs Shallow Copy

The core issue is:

What happens when an object contains a pointer to dynamically allocated memory and we copy the object?

Consider:

```
class Student {
public:
    int age;
    int* marks;

    Student(int a, int m) {
        age = a;
        marks = new int(m);
    }
};
```

Suppose:

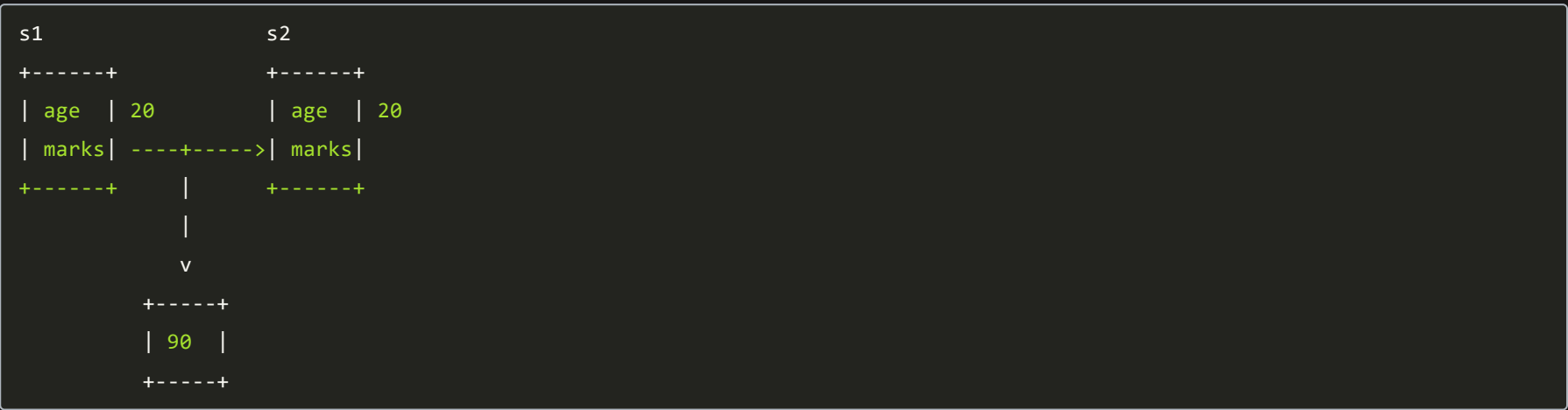
```
Student s1(20, 90);
Student s2 = s1;
```

Since we haven't written a copy constructor, C++ generates a **default copy constructor** for us.

Conceptually, it does:

```
s2.age = s1.age;
s2.marks = s1.marks;
```

So:



Both `s1.marks` and `s2.marks` point to the **same memory**.

That's **shallow copy**.

Why is shallow copy dangerous?

Suppose:

```
*s2.marks = 50;
```

Then:

```
cout << *s1.marks;
```

prints:

```
50
```

because both objects share the same memory.

Even worse, suppose the destructor is:

```
~Student() {
    delete marks;
}
```

Then:

```
Student s1(20, 90);
Student s2 = s1;
```

When the objects are destroyed:

```
s2 destructor → delete marks
s1 destructor → delete marks AGAIN
```

This can cause **double deletion** → **undefined behavior**.

Deep Copy

A **deep copy** creates a completely independent copy of dynamically allocated resources.

We need our own copy constructor:

```
class Student {
public:
    int age;
    int* marks;

    Student(int a, int m) {
        age = a;
        marks = new int(m);
    }

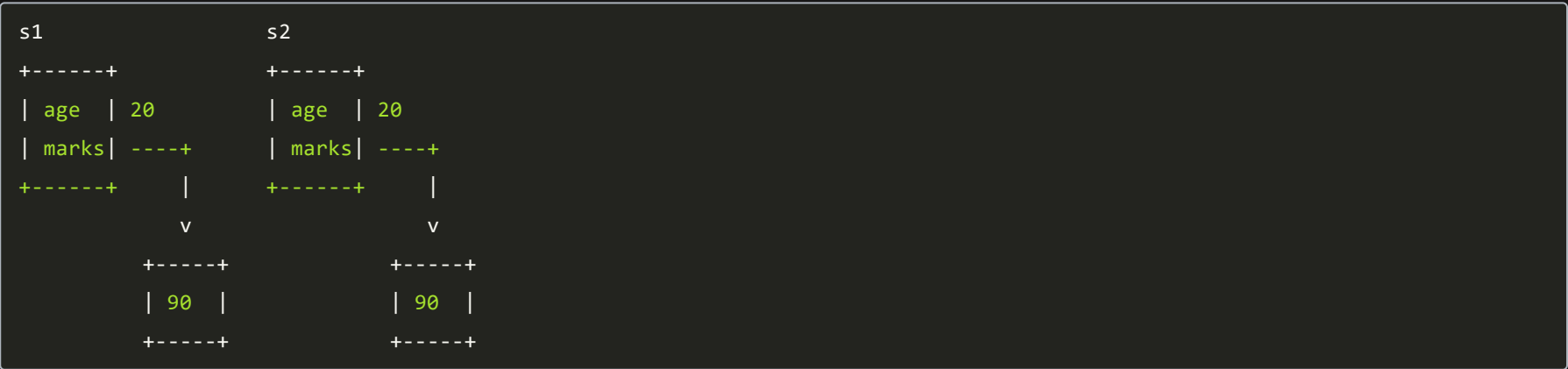
    // Deep copy constructor
    Student(const Student& other) {
        age = other.age;
        marks = new int(*other.marks);
    }

    ~Student() {
        delete marks;
    }
};
```

Now:

```
Student s1(20, 90);
Student s2 = s1;
```

Memory looks like:



Two different memory locations.

Therefore:

```
*s2.marks = 50;

cout << *s1.marks; // 90
cout << *s2.marks; // 50
```

That's a **deep copy**.

What exactly is a Copy Constructor?

A copy constructor is a constructor that creates an object from another object of the same class.

Typical syntax:

```
ClassName(const ClassName& other);
```

For example:

```
Student(const Student& other) {
    age = other.age;
    marks = new int(*other.marks);
}
```

Why `const ClassName&?`

Reference avoids creating another copy.

If we wrote:

```
Student(Student other)
```

then passing `other` itself requires copying `other` → which requires the copy constructor → which requires another copy → infinite recursion.

So we use:

```
const Student& other
```

`const` means we promise not to modify the source object.

When is the copy constructor called?

1. Initialization from another object

```
Student s1(20, 90);

Student s2 = s1;
```

Copy constructor.

Also:

```
Student s2(s1);
```

Copy constructor.

2. Passing an object by value

```
void print(Student s) {
    // ...
}

print(s1);
```

A copy is created for `s`.

3. Returning an object by value

```
Student createStudent() {
    Student s(20, 90);
    return s;
}
```

A copy **may** happen, although modern C++ often uses **copy elision/RVO**, so don't blindly say "return by value always calls the copy constructor."

That's a good interview nuance.

Shallow vs Deep Copy

	Shallow Copy	Deep Copy
Pointer	Copies pointer	Allocates new memory
Resource	Shared	Independent
Modification	Can affect original	Doesn't affect original
Destructor	Can cause double delete	Safe
Implementation	Default copy often does this	Custom copy constructor
Dynamic memory	Dangerous	Properly handled

Interview Example

A classic interview question:

```
What's wrong with this class?
```

```
class A {
    int* p;

public:
    A(int x) {
        p = new int(x);
    }

    ~A() {
        delete p;
    }
};
```

Answer:

The class manages dynamically allocated memory but doesn't define a copy constructor or copy assignment operator. The compiler-generated versions perform shallow copying, so multiple objects can point to the same memory, potentially causing double deletion and unintended shared modifications.

Then implement:

```
A(const A& other) {
    p = new int(*other.p);
}
```

But there's an important next step.

The Rule of Three

Once your class manually manages a resource, defining **only the copy constructor is usually not enough**.

You generally need:

```
class A {
    int* p;

public:
    A(int x) {
        p = new int(x);
    }

    // 1. Copy constructor
    A(const A& other) {
        p = new int(*other.p);
    }

    // 2. Copy assignment operator
    A& operator=(const A& other) {
        if (this != &other) {
            delete p;
            p = new int(*other.p);
        }
        return *this;
    }

    // 3. Destructor
    ~A() {
        delete p;
    }
};
```

These three are the **Rule of Three**:

If a class needs a custom destructor, copy constructor, or copy assignment operator, it probably needs all three.

And in modern C++, this leads to the **Rule of Five** when move semantics are involved:

```
Destructor
Copy constructor
Copy assignment
Move constructor
Move assignment
```

Whenever you see:

```
int* p;  
char* data;  
SomeType* obj;
```

immediately ask:

Who owns this memory, and what happens when the object is copied?

If ownership is involved:

```
Copy  
↓  
Do pointers get copied?  
↓  
Same memory? → Shallow copy  
Different memory? → Deep copy
```

And if you're manually managing resources:

```
Destructor  
+  
Copy constructor  
+  
Copy assignment  
=  
Rule of Three
```

One-line interview answer

Shallow copy copies the pointer itself, so two objects share the same resource. Deep copy creates a separate resource and copies the underlying data, preventing unintended sharing and issues like double deletion.